

# Suivi du projet — Omniscient (MCHub)

Document vivant : décisions prises, ce qui est livré, ce qui reste ouvert, et les idées pour plus tard. À relire/mettre à jour à chaque étape importante plutôt que de repartir de zéro. Anciennement `Deployment Vercel.md` — renommé car son contenu dépasse largement le déploiement initial.

*Dernière mise à jour : 2026-09-24 (visibilité public/privé + invitation, vrai upload d'images, refonte "Mes instances", système de signalement, écran d'accueil du launcher, section abonnements ajoutée).*

---

## 1. État actuel en un coup d'œil

- **Site** : en ligne sur `https://omniscient-theta.vercel.app`, projet Vercel omniscient, branche de production dédiée (`production`, jamais `master` directement — voir §2).
  - **Launcher** : releases publiques sur `nono84250-sudo/omniscient-launcher` (v0.1.8 au moment de cette mise à jour), installateur web (`nsis-web`), mise à jour automatique silencieuse.
  - **Dépôt de dev** : `nono84250-sudo/mchub` (privé) + sauvegarde miroir `nono84250-sudo/mchub-backup-20260921` + zips locaux (`D:\Claude Code\MCHub-backup-*.zip`).
- 

## 2. Décisions de déploiement (prises, toujours valables)

- `mchub` (GitHub) reste privé, reste le dépôt de développement/tests.
  - Second dépôt GitHub public `omniscient-launcher`, releases uniquement (pas de code source) — nécessaire car les Releases d'un repo privé ne sont pas téléchargeables publiquement.
  - Projet Vercel nommé `omniscient` (indépendant du nom du repo GitHub). Nom produit toujours "Omniscient Launcher" côté utilisateur, jamais "mchub"/"launcher".
  - **Séparation des branches** : la Production Branch Vercel est `production`, jamais `master` directement — un commit de test sur `master` ne casse jamais le site en ligne. Promotion = `git push origin master:production`, toujours une action explicite et confirmée, jamais automatique.
  - Deux bases Prisma Postgres séparées : `mchub` (dev + Preview Vercel) et `omniscient-production` (Production uniquement, jamais touchée par les tests).
  - Installateur du launcher : approche **web installer** (`nsis-web` via `electron-builder`) — un petit stub qui télécharge le reste au premier lancement, plutôt qu'un gros `.exe` autonome. Permet la mise à jour automatique et, plus tard, la préconfiguration par client sans rebuild.
  - Fonctionnalité premium (préconfig par serveur) — toujours à **concevoir séparément**, voir §7.
- 

## 3. Checklist d'exécution du déploiement — [OK] tout fait

- [OK] Base `omniscient-production` créée + schéma appliqué.
- [OK] Projet Vercel lié à `mchub` (`rootDirectory: site`), nommé `omniscient`, Production Branch = `production`.
- [OK] Variables d'environnement configurées (Production -> `omniscient-production` ; Preview/Development -> `mchub`).
- [OK] Premier déploiement vérifié (build, auth, DB).
- [OK] Dépôt public `omniscient-launcher` créé.

- [OK] `electron-builder` configuré (`nsis-web`, nom produit "Omniscient Launcher", nom d'artefact **sans espace** — voir §6, un bug de mise à jour auto est venu de là).
  - [OK] Premier build + première release publiée (v0.1.0 -> v0.1.8 au 2026-09-23).
  - [OK] Page de téléchargement sur le site (`/download`), lien direct stable vers `releases/latest/download/...`, version/taille affichées en direct depuis l'API GitHub (cache court + invalidation à la demande après chaque publication).
  - [OK] `electron-updater` : mise à jour automatique **silencieuse** (voir §6 pour l'historique des bugs corrigés).
    - [EN ATTENTE] Système d'abonnement + préconfiguration par serveur pour les clients premium — pas commencé, voir §7.
    - [EN ATTENTE] Nom de domaine + certificat de signature de code — pas commencé, voir §5.
- 

## 4. Fonctionnalités livrées depuis le déploiement initial

- **i18n complet FR/EN** — site (`dictionnaires + t()/getT()`) et launcher (`i18n.js`, dictionnaire propre) ; console de gestion, formulaire/assistant serveur, page Download, console de debug.
- **Migration icônes vers Phosphor** (`lucide-react` retiré) sur tout le site.
- **Nav du site restructurée** : logo + liens principaux à gauche (extensible), langue/connexion à droite.
- **Page Download publique** (`/download`) : détection d'OS côté serveur, cartes Windows/macOS/Linux (mac/Linux "Bientôt disponible", pas encore de build), version/taille en direct.
- **Launcher — installateur web** : stub `nsis-web` (~700 Ko) qui télécharge le paquet complet à l'installation ; nom d'artefact figé, sans espace, sans version (`OmniscientLauncherSetup.exe`) pour que le lien du site et la mise à jour auto restent toujours valides.
- **Launcher — mise à jour automatique silencieuse** (`electron-updater`) : plus d'installateur visible pendant une mise à jour, l'appli se relance seule. Deux bugs réels rencontrés et corrigés en cours de route (voir §6, utile si un symptôme similaire réapparaît).
- **Launcher — bootstrap nettoyé** : barre de titre épurée pendant le démarrage (statut Minecraft/notifications/réduire/fermer masqués, titre centré), plus de délai artificiel.
- **Launcher — console de debug** (fenêtre séparée, `Ctrl+Shift+D` ou Paramètres > Débogage) : logs launcher + Minecraft en direct (jeton d'accès toujours masqué), filtres, recherche, pause/copier/effacer, niveau de log, logs persistés sur disque, rapport de diagnostic, infos système, réparation/vidage du cache, réinitialisation des paramètres.
- **Correctif Java** : la détection exige désormais réellement Java 21 minimum (pas juste "présent + 64 bits") — un vieux Java 8 système faisait planter Minecraft 1.21+ au lancement sans que le launcher ne s'en rende compte.
- **Audit complet du 2026-09-23** (sécurité, qualité de code, UI/UX) + corrections — voir §6.
- **Thème clair du site** (harmonisé avec le launcher, sélecteur Système/Sombre/Clair dans la nav, persisté en `localStorage`) + tokens `--muted2/` `--warn` partagés entre les deux apps.
- **Discord Rich Presence** (`launcher/src/discordPresence.js`) : le launcher annonce sur Discord "Parcourt les serveurs" ou "Joue sur {serveur}" avec le logo Omniscient. Code livré et désactivé proprement tant que `DISCORD_CLIENT_ID` n'est pas configuré — **configuration manuelle restante côté Discord, voir §5.**
- **Connexion Microsoft uniquement** : email/mot de passe entièrement retiré, un seul bouton "Se connecter avec Microsoft" (même identifiant Azure que le launcher). Le profil Minecraft/Xbox (pseudo, UUID) devient la seule identité — voir `site/src/auth.ts/site/src/lib/xboxAuth.ts`.

- **Visibilité des serveurs (Public/Privé)** : bascule dans les Paramètres du serveur, génère un code/lien d'invitation (`omniscient.gg/join/XXXXX`) pour les serveurs privés — masqués de l'annuaire et de l'API publique, rejoignables uniquement via le lien. Réinitialisation du code possible.
  - **Vrai dépôt d'image (glisser-déposer)** : les champs bannière/icône/fond ne sont plus des URLs à coller — upload direct navigateur -> Vercel Blob, conforme à la maquette (dimensions/formats/poids affichés sous chaque zone, limites appliquées côté serveur par champ).
  - **Refonte "Mes instances" du launcher** : grille de cartes (badge Public/Privé/En pause, icône réelle, code d'invitation sur les privées), filtres avec compteurs, et le bouton "Gérer" ouvre enfin la console du site dans le navigateur système au lieu de ne rien faire.
  - **Système de signalement (Reports)** : bouton "Signaler" sur la fiche d'un serveur dans le launcher (type de problème, description, diagnostics de version optionnels) -> nouvel onglet "Signalements" dans la console de gestion du site (Ouverts/Résolus, réponse au joueur, marquage résolu, mise en évidence d'un écart de version). Volontairement limité à la catégorie "Problème technique" — voir §7 pour la catégorie "Comportement/contenu", qui attend le panel admin.
  - **Écran d'accueil du launcher** : tableau de bord personnel à l'ouverture (carte "Continuer" sur le dernier serveur joué, favoris, aperçu "Mes serveurs" si propriétaire) — construit uniquement à partir de données déjà existantes, sans bannière "à la une" ni actualités (voir §7).
- 

## 5. Points ouverts (à trancher)

- [ ] **Nom de domaine** : le client en a-t-il déjà un, ou faut-il en acheter un via Vercel ?
  - [ ] **Clé API CurseForge réelle** : toujours invalide (testé à nouveau le 2026-09-23, l'API répond encore 403). Bloquant pour que la recherche de modpack fonctionne réellement (`site/src/lib/curseforge.ts`).
  - [ ] **LAUNCHER\_API\_KEY partagée** : embarquée dans chaque copie de l'app (extractible techniquement). Acceptable pour l'usage actuel (anti-scraping basique), à surveiller si la sécurité doit monter en exigence.
  - [ ] **Signature de code Windows** : sans certificat, SmartScreen avertit au premier lancement de l'installateur. Pas bloquant, à prévoir si l'image "pro" devient importante.
  - [ ] **Builds macOS/Linux** : la page Download les affiche déjà ("Bientôt disponible") mais aucun build n'existe — seul Windows (`nsis-web`) est construit aujourd'hui.
  - [ ] **Discord Rich Presence — configuration côté Discord** (code déjà livré, voir §4) : je ne peux pas créer l'application Discord ni uploader l'asset à la place de l'utilisateur, ça nécessite son propre compte Discord. Étapes restantes :
    - Créer une application sur <https://discord.com/developers/applications>, la nommer **exactement** "Omniscient Launcher" (c'est ce nom-là, pas une valeur de code, qui s'affiche en gras dans Discord).
    - Onglet "Rich Presence" > "Art Assets" : uploader `launcher/build/icon.png` comme asset, avec la clé exacte `omniscient_logo`.
    - Copier l'Application ID (Client ID, onglet général) et le donner pour l'ajouter à `DISCORD_CLIENT_ID` dans `launcher/.env` avant le prochain build publié.
- 

## 6. Backlog technique (issu de l'audit du 2026-09-23, pas corrigé volontairement)

Corrections déjà appliquées le jour même : fenêtre de connexion Microsoft sans sandbox, SSRF possible via le champ IP d'un serveur, comparaison non constante du secret launcher, nom de fichier externe non assaini, bouton "Effacer" de la console qui ne vidait pas le bon buffer, export mort avec commentaire trompeur ("mode test" disparu), pages Actualités/Notifications en français en dur, barre de titre dupliquée entre les 2 fenêtres du launcher, gestion d'erreur manquante sur les actions de la console de gestion, convention de bouton "plein" incohérente avec le reste du design system, sélecteur de langue inaccessible au clavier, contraste de texte insuffisant (- -muted/- -muted2, surtout en thème clair), unités de taille toujours en français, tokens de couleur dérivés désynchronisés entre site et launcher, rayons de bordure incohérents sur les cartes du site, et plusieurs petits exports/imports morts.

Tout le backlog mineur listé précédemment (nettoyage de Design/, thème clair du site, tokens - -muted2/- -warn partagés, largeur fixe de ManageAvatarMenu, libellés système non traduits) a été traité le 2026-09-23. Reste volontairement en suspens :

- [ ] Colonnes discordUrl/websiteUrl du modèle Server existent en base mais ne sont utilisées nulle part — cohérent avec les liens Discord/site en attente sur la page admin serveur (voir §7), à réutiliser plutôt qu'à dupliquer quand ce chantier démarrera.

---

## 7. Idées pour plus tard (fonctionnalités futures, rien commencé)

- **Panel administrateur plateforme** (site-wide, propriétaire + collaborateurs — distinct de la console "Gérer" par serveur, déjà livrée) : nécessite d'abord un champ role/isAdmin sur le modèle User (n'existe pas encore) avant de construire quoi que ce soit dessus. Couvrirait :
  - Modération/gestion des utilisateurs et serveurs à l'échelle du site.
  - Rédaction des **Actualités** réelles (/dashboard/news existe, vide) — diffusées à tout le monde, et affichées plus tard sur l'écran d'accueil du launcher (bannière "à la une" + flux d'actus, voir plus bas). **Ce n'est pas la même chose que les Notifications** (voir juste en dessous) — l'admin écrit les Actualités une fois pour tout le monde, il n'écrit jamais les Notifications.
  - La catégorie de signalement "**Comportement/contenu**" (harcèlement, triche, contenu offensant) — le dialogue de signalement du launcher n'offre aujourd'hui que "Problème technique" (va au propriétaire du serveur, déjà livré). L'autre catégorie de la maquette doit aller à une boîte de réception modérateur qui n'existe pas encore ; volontairement absente du dialogue pour ne pas avoir un bouton qui ne mène nulle part.
  - La bannière "à la une" (Featured) de l'écran d'accueil du launcher — voir §8, c'est un achat ponctuel, pas une fonctionnalité gratuite du panel lui-même.
- **Notifications personnelles** (bouton cloche déjà en place côté launcher et /dashboard/notifications côté site, mais vide) — **distinctes des Actualités** : personnelles, déclenchées par un événement précis qui concerne le joueur (quelqu'un a répondu à un signalement qu'il a envoyé, une demande d'ami reçue/acceptée), jamais rédigées à la main par un admin. Le système d'amis (phase 1, ci-dessous) en a besoin pour les demandes d'ami — probablement à construire un peu avant le panel admin plutôt qu'après.
- **Système d'amis** (phase 1 déjà scopée dans une session précédente) — demandes d'ami, liste, suppression ; pas de présence/chat en v1. Nécessite une migration DB, prochaine priorité déclarée à un moment donné.
- **Chat et lobby** (phase 2/3, après le système d'amis) : chat (DM et/ou groupe) et un "lobby" où des amis votent un serveur puis lancent le jeu synchronisé ensemble. Les deux ont besoin de la même décision d'infra temps-réel (probablement un service hébergé type Pusher/Ably plutôt qu'un serveur WebSocket auto-hébergé, projet solo).

- **Page publique de serveur personnalisable** ("page builder", façon créateur de site) — remplace la fiche à mise en page fixe actuelle par des blocs personnalisables. Réservée aux abonnements payants, voir §8.
  - **Écran d'accueil du launcher** : la structure existe déjà (voir §4), il ne manque que la bannière "à la une" (§8, achat ponctuel) et le flux d'actualités (ci-dessus, contenu du panel admin) — rien de neuf à construire côté launcher pour ces deux-là, juste du contenu à leur donner.
  - **Statistiques avancées** pour les propriétaires (temps de jeu moyen, pics horaires, joueurs récurrents vs nouveaux) — impliquerait une nouvelle instrumentation (au-delà des compteurs vues/lancements actuels). Candidat pour l'abonnement payant, voir §8.
  - **Domaine personnalisé, signature de code Windows, builds macOS/Linux, clé CurseForge réelle** — toujours en attente, voir §5 (pas des idées nouvelles, juste pas oubliées).
- 

## 8. Système d'abonnements — proposition (rien construit, à valider)

Personne n'avait encore posé de grille précise — jusqu'ici, chaque fonctionnalité premium (page builder, launcher préconfiguré, mise en avant) avait été notée séparément comme "probablement payant" sans dire par rapport à quoi. Ce qui suit est **une proposition à trancher**, pas une décision prise : à ajuster, renommer, refuser en partie.

**Principe de départ** : tout ce qui existe et fonctionne aujourd'hui (fiche serveur, visibilité public/privé, signalements, activité de base) reste gratuit. Rien de ce qui est déjà livré n'est retiré du gratuit pour être remonté en payant — ça casserait la confiance des premiers utilisateurs. Le payant, c'est ce qui vient **en plus**.

Deux mécanismes de monétisation distincts, pas un seul :

- **Abonnement récurrent** (mensuel/annuel) — débloque des capacités permanentes tant qu'il est actif.
- **Achats ponctuels ("boosts")** — indépendants de l'abonnement, à l'unité, pour un besoin ciblé et temporaire (typiquement : la mise en avant).

### Formule Gratuite (toujours, aucune carte bancaire requise)

- 1 serveur publié dans l'annuaire (au-delà : voir Pro).
- Fiche complète : description, bannière/icône/fond (upload), type, version, RAM recommandée.
- Visibilité Public/Privé + lien d'invitation.
- Vue d'ensemble + Activité (vues,ancements, joueurs en ligne instantané).
- **Signalements — toujours gratuit, sans exception.** Le retirer du gratuit inciterait les propriétaires non-payants à ignorer les problèmes de leurs joueurs, ce qui nuit à la qualité de tout l'annuaire, pas juste à eux.
- Duplication, pause, suppression.

### Formule Pro (abonnement payant — nom à définir)

- **Serveurs illimités** (au lieu d'1 seul) — c'est le levier le plus naturel : la plupart des propriétaires n'ont qu'un serveur, ceux qui en gèrent plusieurs sont justement le profil prêt à payer.
- **Page publique personnalisable** ("page builder", voir §7) — mise en page libre au lieu de la fiche à structure fixe.

- **Launcher préconfiguré par serveur** (voir §7) — lien de téléchargement unique, le joueur arrive directement configuré pour ce serveur.
- **Statistiques avancées** (voir §7) — historique étendu, temps de jeu moyen, pics horaires, joueurs récurrents vs nouveaux, export des données.
- **Code d'invitation personnalisé** (ex. play-mcc au lieu d'un code aléatoire à 5 caractères) pour les serveurs privés.
- **Collaborateurs sur un serveur** — inviter quelqu'un d'autre à gérer un serveur (accès à la console "Gérer") sans lui transférer la propriété du compte.
- **Badge "Pro"** visible sur la fiche publique et dans le launcher — signal de confiance, coût de développement quasi nul.
- Limites d'upload plus hautes (images plus lourdes, formats supplémentaires comme un fond animé).

### **Boosts à la carte (achat ponctuel, avec ou sans abonnement Pro)**

- **Mise en avant ("Featured")** sur l'écran d'accueil du launcher, pour une durée fixe (ex. 1 semaine) — c'est l'idée déjà évoquée, qui reste bien un achat séparé et non un palier d'abonnement.
- Possible plus tard : remontée temporaire dans les résultats de recherche/l'annuaire.

### **Pas encore assez défini pour être tarifé**

- **Modération du système d'amis** (bloquer, signaler un abus de demande d'ami) — aucune idée de scope, à revoir une fois la phase 1 des amis en place (voir §7).
- **Chat et lobby** — dépend d'abord de la décision d'infra temps-réel (voir §7) ; trop tôt pour dire si ce sera gratuit, Pro, ou son propre palier.

**Reste à trancher avec le client/l'utilisateur final avant de coder quoi que ce soit ici** : le prix, la limite exacte du gratuit (1 serveur ? plus ?), et si la mise en avant se vend à l'unité ou en pack (ex. 4 semaines). Rien de cette section n'a de schéma de base de données ni de code — c'est uniquement une proposition de répartition des fonctionnalités.